ConcurrentHashMap and Concurrent Collections

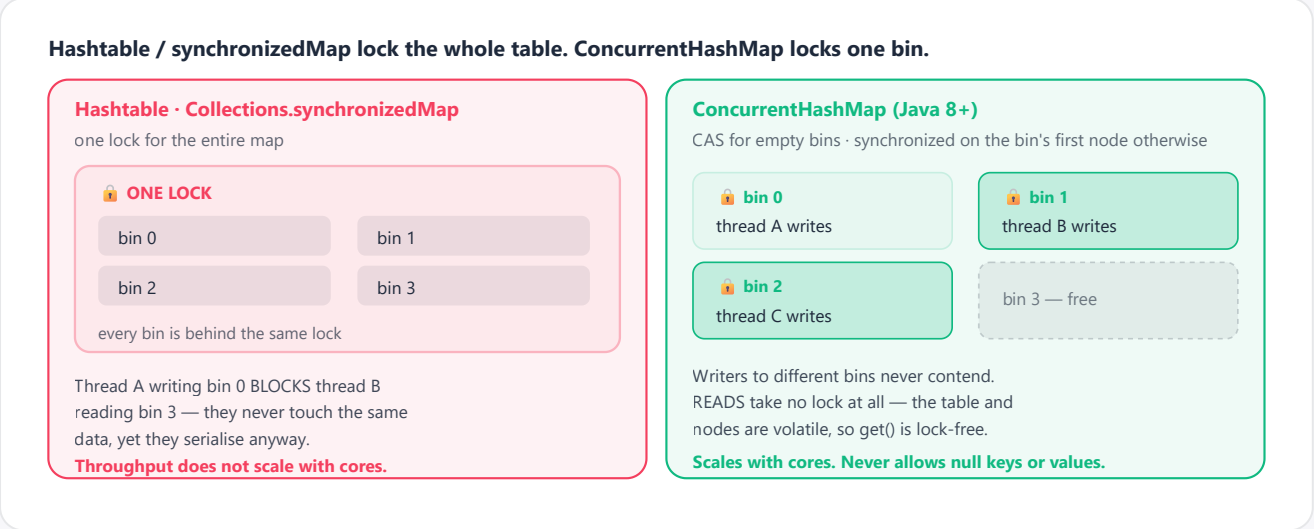
9 min read Updated 28 Jul 2026 Topic 21 of 65

Why it matters

Every server application shares state between request threads, and the collection you pick decides whether that is correct and whether it scales. This topic is also where a lot of candidates reveal a dated mental model: describing `ConcurrentHashMap` in terms of "segments" is describing Java 7, and the implementation changed completely in Java 8. Getting that right is a strong, easily-verified signal.

The concept, simply

There are three ways to make a map usable from several threads, and they are not equivalent:



One lock for the whole table serialises threads that never touch the same data. Locking per bin lets unrelated writers proceed in parallel, and reads take no lock at all.

- `Hashtable` — legacy, every method `synchronized` on the whole object.

- `ConcurrentHashMap` — purpose-built: lock-free reads, and writes that lock only the bin they touch.

How it works

Reads take no lock. The `table` array and each node's `value` and `next` fields are `volatile`, so a reader sees a consistent, up-to-date view without acquiring anything. `get` is therefore lock-free and scales perfectly with cores — which matters enormously, because most real maps are read far more than they are written.

Writes lock one bin. If the target bin is empty, the new node is installed with a single CAS (compare-and-swap) — no lock at all. If the bin is occupied, the thread synchronises on that bin's first node. Two threads writing to different bins never contend.

What changed in Java 8. Java 7 used a fixed number of *segments* (16 by default), each with its own lock — so concurrency was capped at the segment count regardless of table size, and the extra indirection cost memory. Java 8 removed segments entirely in favour of per-bin locking plus CAS, so the effective concurrency scales with the table. It also brought treeification, matching `HashMap`. Saying "segments" describes an implementation that has not existed for a decade.

What it does not give you. Each individual operation is atomic; a *sequence* is not. `if (!map.containsKey(k)) map.put(k, v)` is still a race, because another thread can act between the two calls. That is what the atomic compound methods are for — `putIfAbsent`, `computeIfAbsent`, `merge`, `compute` — and using them is the whole skill here.

Code examples

JAVA

Copy

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();

// ✗ Race: two threads can both read 5 and both write 6. Updates are lost.
```

```java
// ✓ Atomic — the whole read-modify-write happens under the bin's lock.
counts.merge(key, 1, Integer::sum);
counts.compute(key, (k, v) -> v == null ? 1 : v + 1);      // equivalent

// ✓ Atomic "create if missing" — the function runs at most once per key,
//   even under contention, so expensive initialisation is not duplicated.
Map<String, List<String>> index = new ConcurrentHashMap<>();
index.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
// ⚠ but the ArrayList itself is NOT thread-safe — two threads adding to t
//   SAME key's list still race. Use computeIfAbsent(k, x -> new
//   CopyOnWriteArrayList<>()) or ConcurrentHashMap.newKeySet().

// ✓ A counter that scales better than merge under heavy contention:
ConcurrentHashMap<String, LongAdder> adders = new ConcurrentHashMap<>();
adders.computeIfAbsent(key, k -> new LongAdder()).increment();
```

Why `synchronizedMap` is not really a substitute:

JAVA Copy

```java
Map<String, Integer> m = Collections.synchronizedMap(new HashMap<>());

m.put("a", 1);          // ✓ atomic
m.get("a");             // ✓ atomic

// ✗ Iteration is NOT synchronised — you must lock manually, and every
//   other thread blocks for the entire loop.
synchronized (m) {
    for (Map.Entry<String, Integer> e : m.entrySet()) process(e);
}
// Forget that block and you get ConcurrentModificationException at random
// ConcurrentHashMap needs none of this: iteration is weakly consistent
// and never throws.
```

The rest of the concurrent toolkit, and when each is right:

JAVA Copy

```
List<Listener> listeners = new CopyOnWriteArrayList<>();

// Producer/consumer — take() blocks until an element is available, which
// the whole point. This is the backbone of a thread pool's work queue.
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(1000);
queue.put(task);                        // blocks if full → natural backpre
Task t = queue.take();                  // blocks if empty

// Non-blocking queue for high-throughput hand-off:
Queue<Event> events = new ConcurrentLinkedQueue<>();

// Concurrent set — there is no ConcurrentHashSet class:
Set<String> active = ConcurrentHashMap.newKeySet();

// Sorted + concurrent:
ConcurrentNavigableMap<Long, Event> timeline = new ConcurrentSkipListMap<>
```

Under the hood

Why no nulls, revisited with the real reason: in a concurrent map, `get(k)` returning null would be ambiguous between "absent" and "mapped to null", and unlike a `HashMap` you cannot resolve it with a follow-up `containsKey` — the state may have changed in between. Doug Lea's stated position is that allowing null would make correct concurrent code impossible to write, so the class rejects it outright.

`size()` **is an estimate.** Maintaining an exact count would require a single contended counter, which would become the bottleneck the whole design exists to avoid. Instead the map keeps a striped set of counters (the same technique as `LongAdder`) and `size()` sums them — so the answer can be stale the instant you read it. Use it for monitoring, never for control flow. `mappingCount()` is the preferred method, returning a `long` for maps above 2 billion entries.

Iteration is weakly consistent, which is a guarantee rather than an absence of one: an iterator reflects the map at some point at or since its creation, never throws `ConcurrentModificationException`, and each element is returned at most once. It may

When `CopyOnWriteArrayList` is wrong is worth stating, because it is often reached for too eagerly: every single write allocates and copies the entire backing array. For a listener list written once at startup and read constantly, that is perfect. For a list receiving thousands of writes per second, it is a disaster.

Common mistakes

- **Describing "segments".** That is Java 7. Java 8 uses per-bin CAS and synchronisation.
- **Check-then-act on a concurrent map.** `containsKey` then `put` is a race. Use `putIfAbsent`, `merge` or `compute`.
- **Thinking `ConcurrentHashMap` makes the VALUES thread-safe.** A shared `ArrayList` stored as a value is still unsafe.
- **Putting null in it.** Throws by design.
- **Trusting `size()` for control flow.** It is an estimate.
- **Iterating a `synchronizedMap` without an external lock.** Throws `ConcurrentModificationException` at random.
- **`CopyOnWriteArrayList` for a write-heavy list.** Every write copies the whole array.
- **Long or blocking work inside `compute`.** It runs while holding the bin's lock.

Interview lens

> **Q: "How does `ConcurrentHashMap` achieve thread safety?"**
> A: In Java 8 and later, reads take no lock at all — the table and node fields are `volatile`, so `get` is lock-free. Writes lock only the bin they touch: an empty bin is filled with a CAS, and an occupied one is synchronised on its first node. Threads writing to different bins never contend, so throughput scales with cores.

> **Q: "How is that different from Java 7?"**
> A: Java 7 divided the table into a fixed number of segments — 16 by default — each

segments in favour of per-bin locking plus CAS, and added treeification for heavily collided bins.

Q: "`ConcurrentHashMap` vs `Collections.synchronizedMap` vs `Hashtable`?"

A: The latter two put one lock around the whole map, so unrelated threads serialise and reads block writes. They also require you to lock manually around iteration. `ConcurrentHashMap` locks per bin, never locks reads, iterates weakly-consistently without throwing, and offers atomic compound operations. There is no situation today where I would choose `Hashtable`.

Q: "Is `map.get(k) == null ? map.put(k, v) : ...` safe on a `ConcurrentHashMap`?"

A: No. Each individual call is atomic but the sequence is not — another thread can act between them, so you can lose an update or overwrite one. The compound atomic methods exist precisely for this: `putIfAbsent`, `computeIfAbsent`, `compute`, `merge`. Anything read-modify-write must go through one of those.

Q: "Why is `size()` approximate?"

A: An exact size needs a single shared counter, which every write would contend on — becoming the bottleneck the whole design avoids. Instead it stripes the count across several cells, like `LongAdder`, and sums them on demand. So the value can be stale as soon as it is returned. It is fine for metrics and wrong for control flow; `mappingCount()` is the preferred accessor.

Q: "When would you use `CopyOnWriteArrayList`?"

A: When reads massively outnumber writes and iteration must never throw — the classic case is a listener or subscriber list configured at startup. Every write copies

the entire array, so iteration is a lock-free immutable snapshot. It is completely wrong for anything write-heavy.

> **Q: "Producer/consumer between threads — what do you reach for?"**
> A: A `BlockingQueue`, usually `LinkedBlockingQueue` or `ArrayBlockingQueue` with a bounded capacity. `put` blocks when full and `take` blocks when empty, which gives correct hand-off and natural backpressure without any hand-written `wait`/`notify`. Bounding it is the important part — an unbounded queue turns a slow consumer into an `OutOfMemoryError`.

Cheat-card

- **Java 8+: no segments.** Lock-free reads (volatile), per-bin CAS/synchronized writes.
- Individual ops atomic; **sequences are not** → `putIfAbsent` / `computeIfAbsent` / `compute` / `merge`.
- **No null keys or values** — absent vs mapped-to-null would be unresolvable.
- `size()` is an **estimate**; prefer `mappingCount()`. Iteration is **weakly consistent**, never throws.
- Thread-safe map ≠ thread-safe values.
- `CopyOnWriteArrayList` → read-mostly only. `BlockingQueue` → producer/consumer, **bounded**.
- Concurrent set → `ConcurrentHashMap.newKeySet()`. Sorted → `ConcurrentSkipListMap`.
- Never do slow work inside `compute` — it holds the bin lock.

QUICK CHECK — no marks, just for you.

1. How does ConcurrentHashMap achieve thread safety in Java 8 and later?

- ◯ By copying the whole table on every write

Lock-free reads via volatile fields, plus writes that CAS into an empty bin or synchronise on the bin's first node

- By dividing the table into 16 segments, each with its own lock
- By synchronising every method on the map object

Check answer

2. Is 'if (!map.containsKey(k)) map.put(k, v)' safe on a ConcurrentHashMap?

- Yes, as long as the map is declared final
- Only if the value is immutable
- No — each call is atomic but the SEQUENCE is not, so another thread can act between them. Use putIfAbsent, computeIfAbsent, compute or merge
- Yes — ConcurrentHashMap makes all operations on it atomic

Check answer

3. When is CopyOnWriteArrayList the right choice?

- For large lists that are frequently appended to
- For any list accessed from multiple threads
- For producer-consumer hand-off between threads
- When reads vastly outnumber writes and iteration must never throw — a listener list configured at startup is the classic case

Check answer

Previous
Sets: HashSet, LinkedHashSet and TreeSet

Next
Fail-Fast vs Weakly Consistent Iterators

LEARN MORE

ARTICLES & DOCS

- ConcurrentHashMap — API docs
  docs.oracle.com
- java.util.concurrent overview — API docs
  docs.oracle.com

VIDEOS



Java Array vs ArrayList | Collections vs Streams | HashMap vs ConcurrentHashMap | HashMap vs HashSet
Java Guides

Hand-picked references for this lesson.